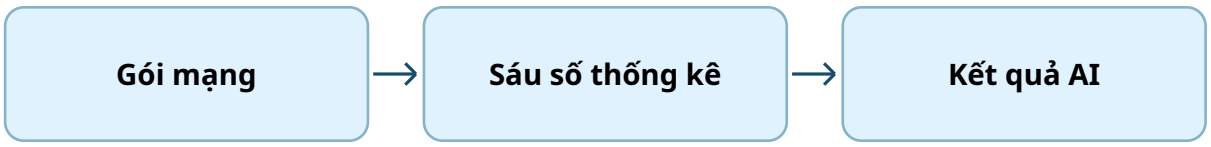
Bắt đầu học AI V6
từ những điều đơn giản

Bản viết lại dễ hiểu • Mỗi bài giải thích một việc

Dùng ví dụ cảm biến gửi dữ liệu, giải nghĩa từng từ rồi mới tới số liệu và công thức.



Bạn chưa cần chạy lệnh. Đọc từng bài, thử trả lời câu hỏi cuối bài rồi mới học tiếp.

Giữ nguyên bộ AI và dữ liệu của bạn. Lần này thay đổi cách giải thích và cách học, không huấn luyện một bộ AI mới.

Ngày viết lại: 08/11/2026. Bản Word để ghi chú; bản HTML chia từng bài và có chỗ bấm thử.

Mục lục

Đọc trang này trước 3

Bài 1. Phần AI của bạn có nhiệm vụ gì? 3

Bài 2. Gói tin, IP và cổng là gì? 4

Bài 3. TCP bắt tay ra sao? “SYN mở” nghĩa là gì? 5

Bài 4. Ba loại lưu lượng khác nhau ở đâu? 6

Bài 5. Dataset của bạn là gì? PCAP và CSV dùng làm gì? 7

Bài 6. Vì sao cần gom một giây và lấy sáu con số? 8

Bài 7. Đọc một dòng dữ liệu thật của bạn 10

Bài 8. Vì sao sáu số lại bị chia và đổi thành q? 11

Bài 9. Model 6–8–3 thực sự làm gì? 13

Bài 10. “Train” là học điều gì? Bo có tự học không? 14

Bài 11. Vì sao accuracy cao mà AI vẫn có thể chưa tốt? 15

Bài 12. Học và thực hành ngay trên gói này 16

Phụ lục A. Khi đã hiểu, hãy xem công thức 18

Phụ lục B. Biết gì trước khi ghép với bạn trong nhóm? 19

Tự kiểm cuối cùng: bạn đã hiểu tới đâu? 21

Bản viết lại dễ hiểu cho phần AI của bạn • 08/11/2026

Đọc trang này trước

Bạn hãy coi tài liệu này như một người đang ngồi bên cạnh giải thích từng bước. Chưa cần học thuộc công thức, đọc Verilog hoặc chạy lại train.

Mỗi bài chỉ tập trung vào một việc. Đọc ví dụ, trả lời câu hỏi cuối bài, rồi xem lời giải ngay bên dưới. Nếu chưa nói lại được bằng lời của mình, hãy đọc lại bài đó; không cần cố đọc hết trong một lần.

• Lần đầu: bài 1–4 để hiểu mạng đang làm gì.

• Lần tiếp theo: bài 5–8 để hiểu file dữ liệu và sáu con số.

• Sau đó: bài 9–12 để hiểu model, train và kết quả.

• Phụ lục A và B: chỉ mở khi muốn xem công thức hoặc ghép phần cứng.

Trong bản HTML, mỗi màn hình là một bài; dùng nút “Bài tiếp theo”. Trong PDF/Word, đọc theo mục lục. Bạn chỉ cần mở PDF để bắt đầu, không cần cài công cụ.

Một điều xuyên suốt: AI của bạn nhìn sáu con số tóm tắt lưu lượng rồi đưa ra một dự đoán. Ta sẽ đi từng bước để hiểu câu này.

Bài 1. Phần AI của bạn có nhiệm vụ gì?

Bắt đầu bằng chuyện cảm biến gửi nhiệt độ

Giả sử cảm biến đo được 30°C và gửi thông tin tới máy chủ MQTT. Máy chủ MQTT là chương trình nhận và chuyển các thông điệp MQTT cho bên cần nhận. Ví dụ này giúp hiểu cách hệ thống hoạt động, không phải khẳng định dữ liệu train của bạn toàn là MQTT.

Dữ liệu đi qua thiết bị gateway của nhóm. Gateway có thể quan sát lượng gói mạng đi qua. Phần AI cần hỗ trợ nhận xét: lưu lượng đang giống bình thường, SYN flood hay vertical scan?

```
Gói mạng đi qua gateway
        v
Đếm và tóm tắt thành sáu con số trong một giây
        v
AI tính toán
        v
Trả về một trong ba tên loại lưu lượng
```

AI không đọc câu “nhiệt độ 30°C” để quyết định tấn công. Model hiện tại nhận các thống kê như số gói và số cổng được truy cập.

Hai từ bạn sẽ gặp thường xuyên

Từ	Hiểu bằng lời thường
Model	Bộ phép tính có các hệ số đã học từ dữ liệu mẫu
Dự đoán	Câu trả lời của model khi nhận dữ liệu mới; có thể đúng hoặc sai

Ví dụ AI trả về SYN flood. Phần điều khiển của gateway còn phải gắn kết quả với nguồn gửi và quy tắc xử lý. Core AI không tự có danh sách IP để chặn.

Cần nhớ: phần của bạn tạo bộ phân loại. Các phần đếm gói, CPU, mạng và điều khiển phải ghép với bộ đó để thành hệ thống.

Tự hỏi: AI nhận cả file PCAP hay sáu con số khi hệ thống chạy?

Lời giải: core AI nhận sáu con số đã đổi sang dạng phù hợp. File PCAP phục vụ chuẩn bị dữ liệu và kiểm thử trên máy tính.

Bài 2. Gói tin, IP và cổng là gì?

Một thông điệp không nhất thiết đi thành một khối duy nhất

Khi hai máy liên lạc, thông tin được truyền qua các gói tin. Bạn có thể hình dung mỗi gói như một phong bì: có người gửi, người nhận và phần thông tin bên trong. Trong mạng thật còn có các gói điều khiển, không mang nội dung ứng dụng như nhiệt độ.

Trong đề cương này, bạn chỉ cần hiểu ba thông tin sau:

Thông tin	Nó trả lời câu hỏi nào?
IP nguồn	Gói được gửi từ địa chỉ nào?
IP đích	Gói đang đi tới địa chỉ nào?
Cổng đích	Gói đang nhắm tới dịch vụ nào trên máy đích?

Ví dụ minh họa: máy A gửi tới IP của máy B, cổng 1883. 1883 thường dùng cho MQTT. Một máy có thể cung cấp nhiều dịch vụ trên nhiều cổng.

IP có thể so với địa chỉ một tòa nhà; cổng giống số phòng dịch vụ trong tòa nhà. Đây là cách nhớ đơn giản, không phải mô tả toàn bộ hoạt động của giao thức mạng.

“Nguồn” và “đích” phụ thuộc chiều gói

Khi cảm biến gửi tới máy chủ, cảm biến là nguồn. Khi máy chủ trả lời, máy chủ là nguồn của gói trả lời đó. Đừng mặc định một IP luôn là nguồn trong mọi dòng.

Máy được chọn để bảo vệ và thống kê trong một mẫu được gọi là **receiver**, nghĩa là máy nhận. Phép đếm của model chỉ lấy gói đi tới IP máy đó trong khoảng đang xét.

Cần nhớ: nhiều cổng nguồn chưa có nghĩa là quét nhiều dịch vụ. Khi tìm vertical scan, cần đặc biệt xem cổng đích.

Tự hỏi: một nguồn gửi tới cổng 22, 80 và 443 trên cùng một IP đích. Nó đã nhắm bao nhiêu cổng đích khác nhau?

Lời giải: ba cổng. Gửi lại cổng 80 nhiều lần không làm số cổng khác nhau tăng thêm.

Bài 3. TCP bắt tay ra sao? “SYN mở” nghĩa là gì?

Hai bên xác nhận trước khi trao đổi dữ liệu

TCP là một cách truyền dữ liệu có kết nối. Trong trường hợp thông thường, hai bên làm ba bước xác nhận trước khi trao đổi dữ liệu của phiên.

Bước	Ai gửi cho ai?	Tên gói	Hiểu bằng lời thường
1	Cảm biến gửi tới máy chủ	SYN	“Tôi muốn thiết lập kết nối.”
2	Máy chủ gửi tới cảm biến	SYN + ACK	“Tôi nhận được yêu cầu; tôi cũng sẵn sàng thiết lập kết nối.”
3	Cảm biến gửi tới máy chủ	ACK	“Tôi đã nhận phản hồi của bạn.”

Quá trình này gọi là **bắt tay TCP**. Sau đó, cảm biến có thể trao đổi dữ liệu MQTT qua kết nối đó.

SYN và ACK là tên các **cờ TCP**. Cờ ở đây là dấu hiệu trong phần điều khiển của gói. “Bật cờ” có nghĩa dấu hiệu đó đang có giá trị 1. ACK dùng để xác nhận đã nhận thông tin; SYN tham gia bắt đầu và đồng bộ kết nối.

Phần AI đếm loại nào?

Trong model của bạn, “SYN mở” là tên gọi cho gói có SYN bật và ACK chưa bật. Nó thường là yêu cầu ở bước 1.

• SYN: được tính vào số SYN mở.

• SYN + ACK: không tính vào số SYN mở.

• ACK: không tính vào số SYN mở.

“Mở” ở đây nghĩa là yêu cầu bắt đầu kết nối. Nó không chứng minh cổng đích đang mở, kết nối đã thành công hoặc kết nối đã thất bại. Muốn biết thành công hay không phải xem diễn biến hai chiều.

Khi nào liên quan tới tấn công?

Một SYN là điều bình thường khi tạo kết nối mới. SYN flood thường làm máy chủ nhận rất nhiều yêu cầu kiểu này; nhiều yêu cầu có thể không hoàn tất bắt tay, gây áp lực lên tài nguyên máy chủ. Nhưng chỉ nhìn một SYN thì không biết đó có phải tấn công hay không.

Cần nhớ: bộ đếm SYN của bạn đếm yêu cầu bắt đầu, không trực tiếp đếm "số kết nối thất bại".

Tự hỏi: thấy SYN-ACK có tăng bộ đếm SYN mở không?

Lời giải: không, vì ACK đã bật. Công thức kiểm hai cờ nằm ở phụ lục A; bây giờ chỉ cần hiểu bảng trên.

Bài 4. Ba loại lưu lượng khác nhau ở đâu?

Normal: bình thường

Ví dụ: cảm biến mở kết nối, gửi dữ liệu theo chức năng rồi tiếp tục hoạt động. Nó có thể gửi SYN và có thể có nhiều gói. Không thể dùng quy tắc "có SYN là tấn công" hoặc "nhiều gói luôn là xấu".

SYN flood: dồn nhiều yêu cầu mở kết nối

Hình dung một dịch vụ liên tục nhận yêu cầu bắt đầu kết nối. Trong ngữ cảnh gây quá tải và các dấu hiệu liên quan, hành vi đó phù hợp SYN flood.

Trong mẫu của bạn, dấu hiệu thường gặp là nhiều gói, phần lớn là SYN mở và tập trung vào một cổng dịch vụ. Đây là dấu hiệu để nhận xét, không phải ngưỡng cố định mà AI luôn làm theo.

Vertical scan: thăm nhiều cổng trên cùng một máy

Ví dụ một nguồn lần lượt thử các cổng 21, 22, 23, 80, 443 trên cùng một IP đích. Nó đang thăm nhiều dịch vụ của cùng máy. Đây là hành vi phù hợp vertical scan khi ngữ cảnh củng cố việc thăm dò.

Nếu thăm cùng một cổng trên nhiều máy, thường gọi là horizontal scan. Model hiện tại không có lớp riêng cho loại đó.

Điều đang quan sát	Có thể nhận xét gì?
Một SYN tới một dịch vụ	Chưa đủ để kết luận tấn công
Rất nhiều SYN tới một dịch vụ	Cần xem xét SYN flood và ngữ cảnh
Một nguồn thăm nhiều cổng trên một máy	Cần xem xét vertical scan
Nhiều người dùng truy cập các dịch vụ khác nhau	Có thể bình thường; tổng nhiều cổng chưa chứng minh một người đang scan

Cả flood và scan đều có thể dùng SYN. Vì vậy AI cần xem nhiều đặc điểm cùng nhau.

Cần nhớ: xem số gói, loại gói, nguồn, đích, cổng và thời gian; không kết luận chỉ từ một dấu hiệu.

Tự hỏi: "1.000 gói" có đủ để chọn Normal, SYN flood hay scan không?

Lời giải: chưa đủ. Cần biết 1.000 gói trong bao lâu, gửi những gì và tới các cổng/đích nào.

Bài 5. Dataset của bạn là gì? PCAP và CSV dùng làm gì?

Dataset là bộ dữ liệu mẫu để học và kiểm tra

Để dạy model, ta cần nhiều ví dụ và thông tin mỗi ví dụ thuộc loại nào. Thông tin loại đó gọi là **nhãn**.

Model của bạn dùng ba mã nhãn:

Nhãn	Tên loại
0	Normal — bình thường
1	SYN flood
2	Vertical scan

0, 1, 2 chỉ là mã tên lớp. Class 2 không có nghĩa nguy hiểm gấp đôi class 1.

Các nguồn bạn đang dùng

Loại	Nguồn dữ liệu
Normal	Phần đã lọc theo nhãn bình thường gốc của IoT-23; danh sách chọn có 28 tệp PCAP
SYN flood	Đoạn SYN flood đã rà soát của capture 34 trong IoT-23
Vertical scan	Mẫu công khai của Chris Sanders mà dự án đã rà soát hành vi quét

IoT-23 là tên một bộ dữ liệu mạng IoT. Chris Sanders là người công bố bộ mẫu packet có file scan đã dùng. Nhóm dùng nguồn gốc rồi chọn/lọc và tạo các biến thể thời gian hoặc trộn nền để học. Không phải toàn bộ IoT-23 đều là SYN flood, và không phải mọi file từ capture malware đều chỉ chứa tấn công.

Ba PCAP/CSV nhỏ trong gói này là **biến thể mô phỏng đã dùng**, lấy từ các nguồn đó. Những ví dụ cảm biến/MQTT ở bài trước chỉ giúp hiểu hệ thống; không khẳng định mọi gói của IoT-23 là MQTT.

PCAP là bản ghi gói, CSV là bảng

File	Bạn xem được gì?
PCAP	Từng gói, thời điểm, IP, cổng và cờ TCP; mở bằng Wireshark
CSV flow	Tóm tắt kết nối/nhóm gói; có thể dùng xem nhãn gốc
CSV sáu đặc trưng	Tóm tắt lưu lượng trong từng giây; đây là loại bảng model cần

Ví dụ minh họa: có 10 gói cùng một kết nối đi tới máy chủ trong một giây. Wireshark có 10 hàng gói. CSV flow có thể chỉ có một hàng kết nối. CSV đặc trưng có một hàng cho máy nhận trong giây đó.

Cần nhớ: một hàng CSV không phải lúc nào cũng là một gói. Hãy xem tên các cột để biết bảng đang ghi loại thông tin gì.

Tự hỏi: file có `packet_count` và `mean_ip_bytes` có khả năng là bảng gì?

Lời giải: bảng đặc trưng đã tổng hợp, không phải bản liệt kê từng packet.

Bài 6. Vì sao cần gom một giây và lấy sáu con số?

AI cần một bảng tóm tắt có độ dài cố định

Một giây có thể có ít gói, giây khác có rất nhiều. Model của bạn không nhận một danh sách packet dài ngắn tùy ý. Ta đếm và tóm tắt chúng thành sáu con số.

Khoảng một giây này được gọi là **cửa sổ**. Ví dụ minh họa cách chia:

```
Từ 10:00:00 đến trước 10:00:01 -> một mẫu thống kê
Từ 10:00:01 đến trước 10:00:02 -> một mẫu khác
```

IPv4 là phiên bản IP dùng trong phép trích này; địa chỉ của nó thường viết như 192.168.1.10. Một mẫu chỉ xét gói đi tới **một IP máy nhận được chọn**. Nếu đếm cả hai chiều hoặc cả nhiều máy, các con số có thể khác và không còn đúng cách model đã học.

Sáu con số trả lời sáu câu hỏi

Tên cột tiếng Anh	Hiểu bằng câu hỏi
packet_count	Có bao nhiêu gói IPv4 hợp lệ đi tới máy này trong một giây?
mean_ip_bytes	Mỗi gói dài trung bình bao nhiêu byte ở phần IP?
syn_ratio_q127	Trong các gói được đếm, SYN mở chiếm tỷ lệ bao nhiêu, viết trên thang 0–127?
src_count	Có bao nhiêu IP nguồn khác nhau gửi tới?
dst_count	Có bao nhiêu IP đích khác nhau được thống kê?
dst_port_count	Có bao nhiêu cổng đích TCP/UDP khác nhau được nhắm tới?

Một byte là đơn vị đo lượng dữ liệu. Độ dài IP dùng ở đây không phải độ dài riêng nội dung nhiệt độ hoặc toàn bộ khung Ethernet. Trong phần mềm, tên trường độ dài đó là IP Total Length.

Vì chọn một IP nhận, `dst_count` thường là 1. Cột này không có nghĩa "có một kẻ tấn công". Số nguồn khác nhau cũng không tự cho biết nguồn nào tốt hay xấu.

Tỷ lệ SYN trên thang 127: tính chậm từng bước

Ví dụ minh họa để học phép tính: một giây có 20 SYN mở, 50 gói TCP khác và 30 UDP. Tổng số gói được đếm là 100.

1. Tỷ lệ SYN là 20 chia 100, bằng 20%.

2. Model cần viết tỷ lệ trên thang 127: lấy 20 chia 100 rồi nhân 127.

3. Kết quả là 25,4. Bỏ phần lẻ còn 25.

Vậy cột `syn_ratio_q127` ghi 25. **25 không phải 25 gói, cũng không phải 25%.** Nó biểu diễn khoảng 20% trên thang 127. Chọn thang này giúp phù hợp dạng số nhỏ mà core dùng.

Mẫu số có cả TCP, UDP và các IPv4 hợp lệ khác tới máy nhận. Chỉ lọc TCP rồi chia sẽ ra một tỷ lệ khác. Các gói phân mảnh hoặc bị thiếu dữ liệu được loại theo quy tắc trích của dự án.

Cần nhớ: sáu con số là sáu thống kê của một máy nhận trong một giây.

Tự hỏi: trong một giây có 10 gói nhưng tất cả từ cùng một IP nguồn, `src_count` là 10 hay 1?

Lời giải: 1, vì đếm số IP khác nhau, không đếm số gói.

Bài 7. Đọc một dòng dữ liệu thật của bạn

Hãy mở ví dụ Normal trước

File `vi_du/normal_train_14_sensor7.csv` có nhiều dòng. Xét dòng có `window_start` là 1800000003. Đây là mốc thời gian của cửa sổ trong file mô phỏng; bạn chưa cần đổi số này sang ngày tháng.

Trong dòng đó có sáu giá trị:

```
14, 54, 18, 1, 1, 1
```

Đừng cố đoán nghĩa theo trí nhớ. Đọc cùng tên cột:

Cột	Giá trị	Đọc thành lời
packet_count	14	Trong một giây có 14 gói tới máy nhận
mean_ip_bytes	54	Độ dài IP trung bình, bỏ phần lẻ, là 54 byte
syn_ratio_q127	18	Tỷ lệ SYN biểu diễn trên thang 127 là 18
src_count	1	Một IP nguồn khác nhau
dst_count	1	Một IP đích khác nhau
dst_port_count	1	Một cổng đích khác nhau

PCAP tương ứng cho biết một giây đó có **hai SYN mở trong 14 gói**. Vì vậy 18 không phải 18 SYN và không phải 18%: hai chia 14 rồi nhân 127, bỏ phần lẻ, được 18.

Nhãn dòng là 0, tức Normal. Đây là ví dụ cụ thể cho thấy dữ liệu bình thường vẫn có SYN.

Đặt ba ví dụ cạnh nhau

Cùng mốc 1800000003, hai file mẫu còn lại có:

Thông tin	Normal mẫu	SYN flood mẫu	Scan mẫu
Nhãn trong CSV	0	1	2
Số gói	14	489	112
Tỷ lệ SYN trên thang 127	18	125	127
Số IP nguồn	1	2	2
Số IP đích	1	1	1

Thông tin	Normal mẫu	SYN flood mẫu	Scan mẫu
Số cổng đích	1	1	70

SYN mẫu: nhiều gói, tỷ lệ SYN cao và tập trung một cổng. Scan mẫu: tỷ lệ SYN cũng cao nhưng có nhiều cổng đích. Hai nguồn trong các mẫu attack có cả thành phần nền đã trộn, không tự chứng minh cả hai nguồn đều đang tấn công.

Đây là ba ví dụ đã có, **không phải quy tắc cố định** như "489 gói luôn là flood".

Các cột khác tạm hiểu như sau

• `label`: nhãn người chuẩn bị dữ liệu ghi, 0/1/2.

• `profile`: tên biến thể dữ liệu, không phải đầu vào AI.

• `window_start`: mốc giây đang xét, không phải thời gian kéo dài của dòng đó.

• `status`: dòng đủ điều kiện để học hay cần xem thêm ngữ cảnh.

• q0 đến q5: sáu số đã thu gọn cho model; học ở bài 8.

`eligible` nghĩa là được cho phép dùng ở bước học/lựa chọn theo quy tắc chuẩn bị dữ liệu. `sparse` là ít gói hoặc ít cổng tấn công nên cần ngữ cảnh. `ambiguous` là đầu vào bị trùng với nhãn khác. Các dòng cần ngữ cảnh vẫn được giữ để xem, không tự đổi thành Normal.

Cần nhớ: có ba việc khác nhau: nhãn file đang ghi gì, packet thể hiện hành vi gì và AI dự đoán gì. AI dự đoán khác nhãn không có nghĩa phải sửa nhãn theo AI.

Tự hỏi: `label = 2` có phải số cổng đích bằng 2 không?

Lời giải: không. 2 là mã lớp vertical scan; số cổng đọc ở `dst_port_count`.

Bài 8. Vì sao sáu số lại bị chia và đổi thành q?

Hiểu "thu gọn số" trước khi học tên chuyên môn

FPGA là chip mà nhóm dùng để thực hiện phép tính bằng mạch số. Khối mạch làm phép tính AI được gọi là core AI. Với core AI nhỏ, ta muốn dùng số nguyên nhỏ thay vì tính số thập phân phức tạp.

Bước chuyển số cho phù hợp cách biểu diễn được gọi là **lượng tử hóa**. Trong bước đầu vào của bạn, nó làm việc sau:

1. Số gói chia 8, bỏ phần lẻ.

2. Độ dài trung bình chia 2, bỏ phần lẻ.

3. Bốn giá trị còn lại giữ theo số đã tính.

4. Nếu giá trị lớn hơn 127 thì chỉ giữ 127.

Làm với dòng Normal vừa đọc

Giá trị trước khi thu gọn	Cách đổi	Giá trị đưa vào AI
Số gói 14	14 chia 8 bằng 1,75; bỏ phần lẻ	1
Mean byte 54	54 chia 2	27
Tỷ lệ SYN 18	Giữ nguyên	18
Số nguồn 1	Giữ nguyên	1
Số đích 1	Giữ nguyên	1
Số cổng 1	Giữ nguyên	1

```
Trước:  [14, 54, 18, 1, 1, 1]
Sau:    [ 1, 27, 18, 1, 1, 1]
```

Sáu số sau được gọi là q0, q1, q2, q3, q4, q5. Phần mềm thường đánh số từ 0 nên q0 là số thứ nhất. q chỉ là tên gọi cho số sau khi thu gọn, không phải sáu nhãn khác.

Thu gọn thì phải chấp nhận mất chi tiết

8 gói và 15 gói đều cho số đầu tiên là 1. 1.016 gói và 2.000 gói đều cho 127 vì đã tới giới hạn. Nếu hai lưu lượng biến thành cùng sáu q, riêng model này không còn thấy sự khác biệt ban đầu đó.

Không tự đổi phép chia 8 thành chia 4 mà giữ model cũ. Khi đổi cách biểu diễn đầu vào, model có thể trả lời khác với ý nghĩa nó đã học.

Chỗ rất dễ nhầm: q0 bằng 0 chưa chắc rỗng

Một giây có 7 gói: 7 chia 8, bỏ phần lẻ, được q0 = 0. Nhưng vẫn có gói, nên vẫn suy luận.

Một giây không có gói nào: bỏ suy luận. Kiểm cửa sổ rỗng bằng **số gói thật**, không bằng q0.

Cần nhớ: model nhận sáu số sau khi đổi, và thứ tự phải luôn giữ nguyên.

Tự hỏi: sáu q có phải sáu quyết định Normal/SYN/scan không?

Lời giải: không. Đó vẫn là sáu đặc trưng đầu vào; quyết định chỉ có sau khi model tính xong.

Bài 9. Model 6–8–3 thực sự làm gì?

Đọc ba số trong tên model

```
6 số đầu vào -> 8 đơn vị tính trung gian -> 3 điểm đầu ra
```

• 6 là sáu q đã học ở bài trước.

• 8 là số đơn vị tính trong lớp giữa, gọi là neuron ẩn.

• 3 là số điểm cho ba lớp: Normal, SYN flood, vertical scan.

Neuron ở đây là một đơn vị tính toán, không phải bộ não nhỏ thật. **Lớp ẩn** nghĩa là kết quả trung gian bên trong model, không phải một loại tấn công bị giấu.

Mỗi đơn vị tính cộng và nhân những gì?

Mỗi neuron lấy các đầu vào nhân với những hệ số rồi cộng thêm một số điều chỉnh. Hệ số gọi là **weight**, số cộng thêm gọi là **bias**. Chúng được điều chỉnh khi train.

Ví dụ chỉ để hiểu phép tính, không phải trọng số model của bạn:

```
Hai đầu vào: 10 và 4
Hai weight:  3 và -2
Bias:       1

Tổng = 10×3 + 4×(-2) + 1 = 23
```

Weight có thể âm hoặc dương. Bias giúp thay đổi mức đáp ứng của neuron ngay cả khi đầu vào bằng 0. Không được nhìn một weight rồi kết luận toàn bộ model luôn làm theo một luật đơn giản.

Trong lớp giữa có bước **ReLU**: phần âm được đưa về 0. Core còn chia/giới hạn giá trị theo cách cố định. Phụ lục A trình bày phép tính đúng của model; hiện tại bạn chỉ cần hiểu nó có chuỗi nhân, cộng và điều chỉnh số.

Cuối cùng AI chọn điểm cao nhất

Với dòng SYN thật, sau khi đổi có sáu q:

```
[61, 20, 125, 2, 1, 1]
```

Model V6 tính được:

Lớp	Điểm model trả về
Normal	718
SYN flood	1142
Vertical scan	-2011

1142 lớn nhất nên chọn SYN flood, tức class 1. Điểm này gọi là **score**. 1142 không phải phần trăm tin cậy; số âm cũng không phải tỷ lệ âm.

Toàn model có 83 tham số học: 72 weight và 11 bias. Tám neuron giữa không được đặt tên sẵn "neuron flood" hoặc "neuron scan"; chúng cùng góp phần tạo ba score.

Cần nhớ: một lần dự đoán là tính bằng hệ số đã có; không phải học lại từ đầu.

Tự hỏi: ba điểm [5,12,3] theo thứ tự Normal/SYN/scan thì chọn lớp nào?

Lời giải: SYN, vì 12 lớn nhất. Không đọc số 12 thành 12%.

Bài 10. "Train" là học điều gì? Bo có tự học không?

Hình dung tập bài có đáp án

Ta chuẩn bị nhiều dòng sáu đặc trưng và nhãn đi kèm. Model thử dự đoán, so với nhãn và điều chỉnh weight/bias để giảm sai. Một phép đo mức sai gọi là **hàm mất mát**. Đây là ý nghĩa cơ bản của train, tức huấn luyện.

```
Sáu số + nhãn tham chiếu
        v
Học các weight và bias trên máy tính
        v
Lưu model đã học
```

Khi có lưu lượng mới, chỉ lấy model đã lưu để tính dự đoán. Bước này gọi là **inference**, nghĩa là suy luận.

Trong hệ thống của bạn, Python train trên máy tính; core FPGA chỉ tính dự đoán. Bo không tự học lại mỗi khi nhận một packet, và khi chạy thật không cần nhập nhãn cho nó.

Vì sao phải tách dữ liệu để đánh giá?

Nếu học một tập câu hỏi rồi chỉ thi lại đúng tập đó, điểm cao chưa chắc đã làm được câu mới. Model cũng vậy.

• **Train:** dùng để học các hệ số.

• **Development:** dùng so sánh các ứng viên và chọn cấu hình.

• **Test độc lập:** dữ liệu giữ riêng, không dùng để điều chỉnh/chọn model; nên có phiên thu mới với nhãn được kiểm.

Trong V6, các biến thể vẫn từ cùng nguồn SYN và scan ban đầu. Có 120 biến thể không có nghĩa có 120 phiên tấn công độc lập. Đổi thời gian, chia đoạn hoặc lặp lại dữ liệu không tạo hành vi nguồn mới.

Dữ liệu gốc có nhiều Normal hơn attack. Quy trình học đã cân bằng số mẫu đầu vào khác nhau giữa các lớp bằng cách lặp mẫu của lớp ít. Lặp giúp cách học cân bằng hơn nhưng không biến thành dữ liệu thực mới.

Bản V6 hiện đã có trọng số. Bạn không cần train lại để đọc tài liệu này hoặc học cách suy luận.

Cần nhớ: điểm trên dữ liệu đã dùng để phát triển model không phải lời bảo đảm cho mọi mạng mới.

Tự hỏi: chép một PCAP thành 100 file có được 100 phiên thu độc lập không?

Lời giải: không. Vẫn cùng dữ liệu và hành vi ban đầu, dù tên file hoặc thời gian có thay đổi.

Bài 11. Vì sao accuracy cao mà AI vẫn có thể chưa tốt?

Accuracy là tỷ lệ trả lời đúng chung

Giả sử có 1.000 ví dụ: 990 Normal, 10 tấn công. Một chương trình luôn nói “Normal” vẫn đúng 990/1.000, tức 99%. Nhưng nó bỏ sót cả 10 tấn công.

Đây là ví dụ giả định để hiểu tỷ lệ, không phải số liệu đánh giá của V6. Nó cho thấy cần hỏi thêm: phát hiện được bao nhiêu attack và báo nhầm bao nhiêu Normal?

Xem số liệu thật của model bạn

Trong thống kê gốc của V6:

• Có 21 cửa sổ SYN có nhãn SYN; model gọi đúng cả 21.

• Có thêm 12 cửa sổ Normal bị model gọi nhầm là SYN.

• Như vậy model nói “SYN” tổng cộng 33 lần, nhưng chỉ 21 lần đúng theo nhãn hiện có.

Precision SYN trả lời: “Trong các lần AI nói SYN, có bao nhiêu lần đúng?” Ở đây là 21 chia 33, khoảng 63,64%.

Recall SYN trả lời: “Trong các SYN có nhãn, AI tìm được bao nhiêu?” Ở nhóm này là 21 chia 21, bằng 100%.

Đây vẫn là thống kê trên dữ liệu đã tham gia phát triển model, không phải phép kiểm mạng mới độc lập. Tập gốc có rất nhiều Normal nên accuracy chung gần 100% không xóa được vấn đề báo nhầm.

Model còn những lỗi nào?

Nhóm kiểm	Điều cần hiểu
Dữ liệu gốc	Còn 12 Normal bị gọi SYN; trong đó có các đầu vào trùng nhau giữa nhãn Normal và SYN
Development cùng nguồn	Có một cửa sổ SYN 803 gói bị dự đoán thành Normal
Lab cũ	Đúng 218/219; có một Normal bị gọi scan

Nếu hai mẫu mang nhãn khác nhau nhưng sau thu gọn có đúng cùng sáu số, model chỉ thấy cùng đầu vào nên phải trả cùng kết quả. Cần xem lại ngữ cảnh/nhãn/cách biểu diễn, không tự sửa nhãn để báo cáo đẹp.

"PASS RTL" là chuyện khác

RTL là mã mô tả mạch số. Bài kiểm 3.762 vector cho thấy core tính giống kết quả tham chiếu trong các trường hợp kiểm. "Vector" ở đây là một bộ số đầu vào.

Điều đó giống kiểm máy tính có nhân cộng đúng không. Nó không tự chứng minh dữ liệu/nhãn hoàn hảo hoặc model sẽ phát hiện tốt trên mọi mạng mới. Kiểm timing và chạy bo lại là những bước triển khai khác.

Cần nhớ: phân biệt tính đúng phép toán với phân loại đúng lưu lượng.

Tự hỏi: AI nói SYN 33 lần, trong đó 12 lần là Normal. Có được nói "100% dự đoán SYN đúng" không?

Lời giải: không. Precision SYN chỉ khoảng 63,64% trong nhóm số liệu này, dù recall SYN là 100%.

Bài 12. Học và thực hành ngay trên gói này

Việc đầu tiên: tập nói bằng lời

Đọc lại dòng Normal [14,54,18,1,1,1] và nói: "Trong một giây có 14 gói tới một máy nhận, từ một nguồn, nhắm một cổng. Mean IP là 54 byte; điểm tỷ lệ SYN là 18 trên thang 127."

Nếu nói được và giải thích vì sao 18 không phải 18 gói, bạn đã hiểu phần nền quan trọng.

Bước 1 — Xem bảng rút gọn

Mở bản HTML của tài liệu và tới bài 7, hoặc mở `vi_du/BANG_TAP_DOC.csv`. Bảng rút gọn chỉ có ba dòng mẫu Normal/SYN/scan để tập đọc. File CSV gốc với các cột tiếng Anh vẫn được giữ cạnh đó.

Nếu Excel mở CSV thành một cột, bạn có thể xem bảng trong PDF/HTML trước; chưa cần sửa Excel để học. Không dùng bảng ba dòng này thay toàn bộ dữ liệu train.

Bước 2 — Mở từng PCAP

Các file sau đã được giải nén sẵn, không cần chạy Python:

```
vi_du/pcap_de_mo/1_Binh_thuong.pcap
vi_du/pcap_de_mo/2_SYN_flood.pcap
vi_du/pcap_de_mo/3_Vertical_scan.pcap
```

Mở Wireshark File Open chọn file. Mỗi hàng ở màn hình là một frame/gói đã ghi. Chọn một hàng, xem phần IPv4 để biết nguồn/đích và phần TCP để xem cổng/cờ.

Chưa cần hiểu toàn bộ các mục Wireshark hiển thị. Trước mắt chỉ tìm: Source, Destination, TCP destination port và SYN/ACK.

Bước 3 — Xem đúng một giây

Để khớp các dòng đã đọc ở bài 7, nhập vào ô display filter của Wireshark rồi nhấn Enter:

```
frame.time_epoch >= 1800000003 && frame.time_epoch < 1800000004
```

Dấu && nghĩa là "và": lấy thời điểm từ số bên trái tới trước số bên phải. Sau lọc, các file mẫu cho lần lượt 14, 489 và 112 frame. Đây là những PCAP mô phỏng đã chỉ giữ hướng tới receiver. Nó không phải tổng frame của cả file.

Để chỉ xem SYN mở trong giây đó, dùng:

```
frame.time_epoch >= 1800000003 && frame.time_epoch < 1800000004 && tcp.flags.syn == 1 &&
 tcp.flags.ack == 0
```

Khi dán, có thể viết thành một dòng. Ba file lần lượt có 2, 483 và 112 SYN mở trong cửa sổ đó. File một chiều không có đủ phản hồi để chứng minh lại bắt tay của từng kết nối; muốn điều tra điều đó phải xem nguồn có cả hai chiều.

Bước 4 — Bấm thử trong HTML

Ở bài 9 có nút Normal, SYN flood và Scan. Bấm từng nút, xem ba score và tên lớp thắng. Muốn xem số trước/sau khi thu gọn thì mở "Xem sáu số trước và sau".

Bấm thêm ví dụ SYN 803 ở bài 11 để thấy một lỗi thật đã ghi. Công cụ giữ nguyên model, không sửa trọng số và không phát traffic.

Bước 5 — Nói lại phần của mình trong một phút

"Em dùng dữ liệu Normal, SYN flood và vertical scan. Em trích sáu thống kê cho một máy nhận trong một giây, đổi chúng thành số nguyên nhỏ rồi đưa vào model 6–8–3. Model trả ba score và chọn lớp có điểm cao nhất. Em kiểm phép tính giữa phần mềm và core FPGA, đồng thời ghi rõ những lỗi phân loại còn lại."

Tự hỏi: đã hiểu sáu số và ba score thì có cần học thuộc toàn bộ weight để trình bày không?

Lời giải: không. Cần biết chúng ở đâu, được học như thế nào và cách kiểm một ví dụ. Phụ lục dưới giúp đọc chi tiết khi bạn đã vững phần cơ bản.

Phụ lục A. Khi đã hiểu, hãy xem công thức

A1. Các tên trong code chỉ điều gì?

Tên	Nghĩa
N	Số gói thật được đếm trong một giây
B	Tổng độ dài IP của các gói đó
S	Số SYN mở trong các gói đó
raw	Sáu đặc trưng trước khi chia/giới hạn cho core
q	Sáu số sau chia/giới hạn
floor hoặc dấu //	Với số không âm ở đây: chia rồi bỏ phần lẻ
clip	Giữ số trong một khoảng; vượt biên thì lấy giá trị biên

Dòng Normal có N = 14, B = 768, S = 2:

```
Mean byte = 768 chia 14, bỏ phần lẻ = 54
Tỷ lệ SYN = 127 × 2 chia 14, bỏ phần lẻ = 18
raw = [14,54,18,1,1,1]
q   = [1,27,18,1,1,1]
```

Chú ý khi gọi hàm C: nó nhận **N, B, S**, tức số gói, tổng byte và số SYN thực. Nó tự tính mean và tỷ lệ. Không truyền 54 vào chỗ tổng byte 768 hoặc 18 vào chỗ số SYN 2.

A2. Công thức đếm SYN mà bài trước làm bạn khó hiểu

```
(flags & 0x12) == 0x02
```

`flags` là nhóm dấu cờ trong gói TCP. Trong cách biểu diễn của TCP, SYN có vị trí ứng với 0x02, ACK có vị trí ứng với 0x10. Dấu & ở đây là phép AND theo bit: & `0x12` chỉ lấy hai vị trí SYN và ACK để kiểm.

Nếu kết quả bằng 0x02 thì SYN bật, ACK tắt. Đó là điều kiện đã nói bằng lời ở bài 3. Công thức này chỉ kiểm hai cờ đó, không bắt buộc mọi cờ khác bằng 0.

Gói	Sau khi chỉ giữ hai cờ	Có bằng 0x02 không?
SYN	0x02	Có, được đếm
SYN + ACK	0x12	Không
ACK	0x10	Không

Bạn không cần thuộc số hex để hiểu hành vi TCP. Chỉ khi đọc code kiểm cờ mới cần đối chiếu bảng này.

A3. Tính một neuron thật

Với mẫu SYN, sáu q là [61,20,125,2,1,1]. Neuron ẩn đầu tiên dùng các weight [-25,3,40,-127,2,-35] và bias -1135.

```
Tổng = -25×61 + 3×20 + 40×125 - 127×2 + 2×1 - 35×1 - 1135
     = 2113
```

Core dịch phải số học 5 bit, tương đương chia 32 rồi lấy phần nguyên xuống, sau đó giữ trong 0–127. Với số dương 2113, kết quả là 66. Với số âm, phải giữ đúng quy tắc dịch có dấu, không tùy tiện thay bằng phép chia C.

Làm đủ tám neuron được [66,0,11,97,0,0,1,0]. Nhân–cộng tiếp bằng lớp đầu ra cho [718,1142,-2011], nên chọn SYN.

Weight dùng số nguyên có dấu 8 bit; bias và bộ cộng dồn dùng 32 bit. Score trả về là số nguyên có dấu 16 bit. Bit là một chữ số nhị phân 0/1; số bit quyết định phạm vi biểu diễn. Giới hạn số nhằm làm phép tính nhỏ và thống nhất giữa Python/C/RTL.

A4. 6–8–3 có 83 tham số ra sao?

• Sáu đầu vào tới tám neuron: 6×8 = 48 weight.

• Tám neuron có tám bias.

• Tám kết quả tới ba output: 8×3 = 24 weight.

• Ba output có ba bias.

Tổng 48+8+24+3 = 83. Tệp `model/model.json` ghi các số này; model số nguyên có w1 gồm 8 hàng × 6 cột và w2 gồm 3 hàng × 8 cột. Mỗi hàng thuộc một neuron đầu ra của lớp đó.

Phụ lục B. Biết gì trước khi ghép với bạn trong nhóm?

B1. Người nhận cần thống nhất đầu vào và đầu ra

Clock là nhịp đồng hồ điều khiển mạch; một chu kỳ là một nhịp. Reset là yêu cầu đưa core về trạng thái bắt đầu.

Sáu q được xếp trong một nhóm 48 bit, mỗi số một byte. Số thứ nhất ở byte thấp nhất. Đây là đóng gói số để truyền qua giao diện, không có nghĩa model có 48 đặc trưng.

Các tên tín hiệu chính:

Tín hiệu	Hiểu bằng lời
feat	Sáu q đã đóng gói
start	“Hãy bắt đầu tính mẫu này”
busy	“Core đang tính”

Tín hiệu	Hiểu bằng lời
done	“Đã có kết quả mới”
class_id	Mã lớp 0/1/2
scores	Ba score có dấu

Gửi start một chu kỳ khi core rảnh. Nếu gửi lúc busy, core bỏ qua yêu cầu mới đó. done chỉ lên một chu kỳ nên phía nhận phải bắt đúng dấu hiệu. Reset mức thấp của core được thực hiện ở cạnh lên clock.

B2. Thời gian tính không phải toàn bộ thời gian phát hiện

Core cần 84 chu kỳ từ lúc nhận start tới done theo bài kiểm. Ở clock giả định 27 MHz, riêng phép tính khoảng 3,11 microsecond.

Nhưng dữ liệu được gom trong một giây, rồi còn thời gian đọc counter, giao tiếp và quyết định. Không nói hệ thống phát hiện/chặn trong 3,11 microsecond chỉ từ số chu kỳ core. Chạy trên bo phải kiểm riêng cả hệ thống.

B3. Các file nên đọc sau

File trong gói AI bàn giao	Đọc khi muốn biết
model/model.json	Trọng số, số neuron và cách chia/giới hạn
tools/ai_reference.py	Phép tính tham chiếu bằng Python
firmware/ai_quant_v6.h	Counter đổi sang q và 48 bit thế nào
rtl/ai_core_model_v6.v	Cách mạch tính lần lượt từng neuron
HOP_DONG_GHEP_AI_V6.md	Giao tiếp để ghép vào hệ thống
model/metrics.json	Số liệu và lỗi phân loại

Tên/đường dẫn trên thuộc gói AI bàn giao, không phải mọi file đều nằm trong gói học dễ hiểu này. Gói học có model, dữ liệu mẫu và công cụ Python học thêm; bộ bàn giao cho bạn tích hợp vẫn giữ riêng.

Bản kỹ thuật cũ trong `tham_khao/DE_CUONG_KY_THUAT_V6.pdf` dành để tra cứu sau. Nó không phải tài liệu cần đọc đầu tiên.

Tự kiểm cuối cùng: bạn đã hiểu tới đâu?

Hãy trả lời bằng lời của mình, rồi đối chiếu:

1. SYN có phải luôn là tấn công? Không, đó còn là bước mở kết nối bình thường.

2. Một dòng feature CSV mô tả gì? Một máy nhận trong một cửa sổ một giây.

3. `src_count = 3` nghĩa gì? Có ba IP nguồn khác nhau, không phải ba gói.

4. Điểm tỷ lệ SYN 18 có phải 18 gói? Không; là tỷ lệ trên thang 127.

5. 6–8–3 nghĩa gì? Sáu đầu vào, tám neuron giữa và ba score.

6. Ai học các weight? Chương trình train trên máy tính; core trên bo dùng chúng để dự đoán.

7. Ba score có phải phần trăm? Không; chọn score lớn nhất theo quy ước model.

8. PASS phép tính có nghĩa nhận mọi tấn công mới? Không; cần đánh giá trên dữ liệu mới độc lập.

Bạn có thể chia việc học thành ba lượt: bài 1–4, bài 5–8, bài 9–12. Chỉ đọc phụ lục khi các câu trên đã rõ. Nếu còn vướng, ghi đúng từ hoặc dòng chưa hiểu để học tiếp từ chỗ đó.

Số liệu và ví dụ được giữ theo model V6 đã bàn giao; lần viết lại này không train lại. Ba mẫu dùng nguồn IoT-23/Chris Sanders đã biến đổi có kiểm soát, không phải phiên thu mới. Ghi nguồn và checksum chi tiết trong `VI_DU_DA_DOI_CHIEU.json`; nguồn bài học được đối chiếu với gói ZIP bạn gửi.